Project Documentation: SAM for Life

This document provides a comprehensive technical overview of the **SAM for Life** web application, detailing the technology stack, project architecture, key features, database models, performance optimizations, and deployment configurations.

---

1. Technical Stack Overview

The application is built using a modern decoupled architecture: a React client communicating with a Node.js Express API.

Frontend (Client)

* **Core Library:** React 19 (Single Page Application)
* **Routing:** React Router Dom v7 (for routing and client-side page transitions)
* **Styling:** Tailwind CSS (for modern utility-first responsive layout and components)
* **Icons:** Lucide React
* **UI Components:** Radix UI primitives (Accordion, Dialog, Separator, etc.)
* **API Client:** Axios (configured with interceptors to inject authorization headers)
* **PDF Engine:** `jsPDF` (for dynamic, client-side download of donation receipts and invoices)
* **Excel Engine:** `xlsx` (SheetJS, for exporting user forms into formatted Excel spreadsheets)
* **Build tool:** Craco (Create React App Configuration Override)

Backend (Server)

* **Runtime:** Node.js
* **Framework:** Express (REST API endpoints)
* **Database Driver:** Mongoose (MongoDB ODM)
* **Email Engine:** Nodemailer (SMTP integration pointing to IONOS mail server)
* **Security:** `bcryptjs` (password hashing) & `jsonwebtoken` (JWT secure session tokens)
* **Payment Processor:** Stripe SDK (for processing payments, with a built-in dummy simulation fallback)
* **Deployment Platform:** Vercel (configured as serverless functions via `vercel.json`)

---

2. Project Architecture

Directory Layout

```text
sam-for-life-second-main/
%%%  backend/                  # Express API Server
%    %%%  .env                 # Environment configurations (SMTP, Stripe, Mon
%    %%%  db.js                # Mongoose schemas, connections, and JSON DB
%    %%%  mailer.js            # SMTP transporters, HTML wrappers, and email
%    %%%  routes.js            # Express API routes (CMS, Auth, Payments, For
%    %%%  seed.js              # Database seeding and automatic migration scri
%    %%%  server.js            # Express app config and entry point (exported f
%    %%%  vercel.json          # Vercel serverless function bindings
%%%  frontend/                 # React SPA Client
%%%  public/                 # HTML template, sitemap.xml, and robots.txt
%%%  src/
%    %%%  api/client.js      # Axios client and CMS/Auth endpoints
%    %%%  components/        # Layouts, navbar, footer, and utility components
%    %%%  contexts/         # AuthContext and ContentContext (SWR Caching)
%    %%%  lib/              # Client utilities (pdf.js and utils.js)
%    %%%  pages/            # Public pages (Home, About, Stories, Donate, etc.)
%    %%%  App.js            # Router config and root layout
```

---

3. Database Schema Models

The database uses MongoDB. If `MONGO_URL` is omitted, the backend automatically falls back to storing data locally in a `db.json` file.

SiteSettings (Singleton)

Stores all customizable website copy, address details, and email notification templates.
* `hero_badge`, `hero_headline_a`, `hero_headline_b`, `hero_subheadline`, `hero_image`
* `mission_eyebrow`, `mission_title`, `mission_body`
* `about_intro_body`, `about_story_body` (Supports split paragraphs)
* `receipt_charity_number`, `receipt_address`, `receipt_thank_you` (for PDFs)
* **Email Templates:** Store custom subjects and HTML bodies for all public forms:
* `email_contact_admin_subject`, `email_contact_user_subject`, `email_contact_user_body`
* `email_volunteer_admin_subject`, `email_volunteer_user_subject`, `email_volunteer_user_body`
* `email_partnership_admin_subject`, `email_partnership_user_subject`, `email_partnership_user_body`
* `email_fundraise_admin_subject`, `email_fundraise_user_subject`, `email_fundraise_user_body`
* `email_donation_admin_subject`, `email_donation_user_subject`, `email_donation_user_body`

PaymentTransaction

Tracks all checkout attempts, stripe status values, and donor info.
* `session_id` (String, Unique)
* `amount` (Number)
* `currency` (String, default: `'gbp'`)
* `frequency` (String: `'one-time'` or `'monthly'`)
* `donor_name` / `donor_email` (Strings)
* `payment_status` (String: `'initiated'`, `'paid'`, etc.)
* `receipt_sent` (Boolean, default: `false` - tracks whether notification was dispatched)

Dynamic Collections

* **Story:** Individual youth success profiles (`name`, `age`, `role`, `quote`, `body`, `image`).
* **NewsItem:** Blog posts (`date`, `tag`, `title`, `desc`, `published`).
* **TeamMember:** Team profiles (`name`, `role`, `bio`, `initials`).
* **Value:** Organizational core principles.
* **ProgrammeStep:** The 3-stage guide details.
* **InvolvementCard:** Landing links on the Get Involved page.

---

4. Key Custom Features

1. CMS Email Template Editor & SMTP

* **Nodemailer SMTP:** Connected to `smtp.ionos.co.uk` on port `587`.
* **CMS customizer:** Admin can update email subjects and body messages dynamically in the Settings editor.
* **Placeholder replacement:** Templates support parsing standard tags:
* Forms: `{name}`, `{email}`, `{subject}`, `{message}`, `{company}`, `{phone}`
* Donations: `{name}`, `{amount}`, `{frequency}`, `{transaction_id}`

2. Client-Side PDF Generator

Inside the transactions log, admins can click **Download Receipt** or **Download Invoice**.
* Powered by `jsPDF`.
* Pulls dynamic charity details (charity number, address, custom message) from `SiteSettings` dynamically.
* Generates on-brand PDF documents client-side with clean layouts, payment status indicators, and grids.

3. Excel Submission Export

The Inbox page features a primary **Export to Excel** button.
* Powered by `xlsx` (SheetJS).
* Automatically queries all database submissions and downloads a single spreadsheet workbook with **5 separate sheets** representing each form submission collection.

---

5. Performance and SEO Optimizations

1. Database Parallelization

In the `/api/cms/all` route, sequential Mongoose queries were optimized to run concurrently using `Promise.all()`, reducing the backend data-fetch latency from ~600ms to ~140ms.

2. Vercel CDN Edge Caching

The `/api/cms/all` API route sets CDN cache headers:
```javascript
res.setHeader('Cache-Control', 's-maxage=10, stale-while-revalidate=3600');
```
Vercel Edge servers cache the CMS JSON response. Repeat visitors fetch data directly from Vercel's global CDN Edge nodes in **<20ms**, bypassing cold starts and database queries entirely.

3. Frontend Stale-While-Revalidate (SWR)

In the frontend `ContentProvider`, data is stored in `localStorage` on initial fetch. On subsequent page visits, the cached data renders **instantly** (eliminating the initial loading spinner), and the UI silently fetches updates and writes them back to the cache in the background.

4. Image Compression

Dynamic Unsplash image resizing parameters (`&w=800&q=80&auto=format`) are appended to all CMS-uploaded URLs, compressing raw 4-5MB images down to ~95KB without visual degradation.

5. SEO Best Practices

* **robots.txt:** Restricts indexing of private admin pages and maps the sitemap.
* **sitemap.xml:** Lists all public URLs to guide crawlers.
* **Charity Schema.org Markup:** Inject JSON-LD structured data in the `<head>` of `index.html` to establish domain context and local brand trust with Google.

---

6. How to Run Locally

1. Start Backend Server

1. Navigate to the `backend/` directory.
2. Ensure you have a `.env` file containing:
* `MONGO_URL`
* `STRIPE_SECRET_KEY`
* `SMTP_USER` / `SMTP_PASS`
3. Run:
```bash
npm install
npm run dev
```

2. Start Frontend Client

1. Navigate to the `frontend/` directory.
2. Run:
```bash
npm install --legacy-peer-deps
npm start
```
3. Open `http://localhost:3000` in the browser.